Department of Electrical and Infromation Engineering

14/05/2026

Threat Containment Strategies for Compromised xApps in Open RAN

Undergraduate Project Progress

Supervisor: Dr. Chatura Seneviratne
Co-Supervisor: Prof. Dr. An Braeken
Co-Supervisor: Mr. Pramitha Fernando

Dewmith M.K.J. - EG/2021/4474
Dilshan M.D.K. - EG/2021/4485
Sewinda L.L.D. - EG/2021/4807
Thilohith T.K. - EG/2021/4832

Contents

Introduction

The foundational transition from traditional RAN systems to the O-RAN, **Problem Statement**, **Objectives** and **Scope** of the threat containment framework

Background

Introduction 01

- Evolution of the RAN: Transition from **monolithic, vendor-locked proprietary hardware** to **open**, **disaggregated**, and **software-based components**.
- Architectural Disaggregation: Breaking the 5G base station (gNB) into logical units: **O-CU** (O-RAN Central Unit), **O-DU** (O-RAN Distributed Unit) and **O-RU** (O-RAN Radio Unit).
- Open Interfaces: Integration of multi-vendor components facilitated by **standardized interfaces like E2, A1, O1 and O2**. **[1]**

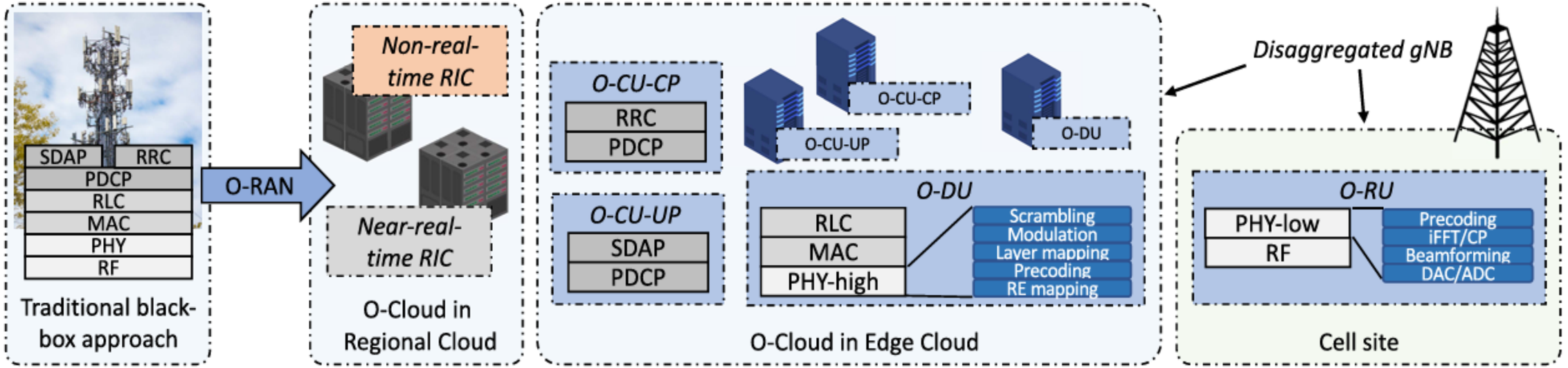


Figure 01: Evolution of traditional base station to a virtualized gNB with disaggregation [1]

Problem Statement

Introduction 01

- Because O-RAN is open, third-party xApps can now execute control actions directly within the Near-RT RIC.
- A single malicious xApp can misuse **shared resources**, move **laterally** to attack other components, or **manipulate network functions** to leak private user data.
- While current O-RAN security protects the "doors" (**interfaces**), it ignores **runtime isolation** and **continuous authentication** of the xApps themselves. This creates a critical security gap in multi vendor environments.

Table 01: Key Issues Mentioned in Oran-Alience WG11 **[6]**

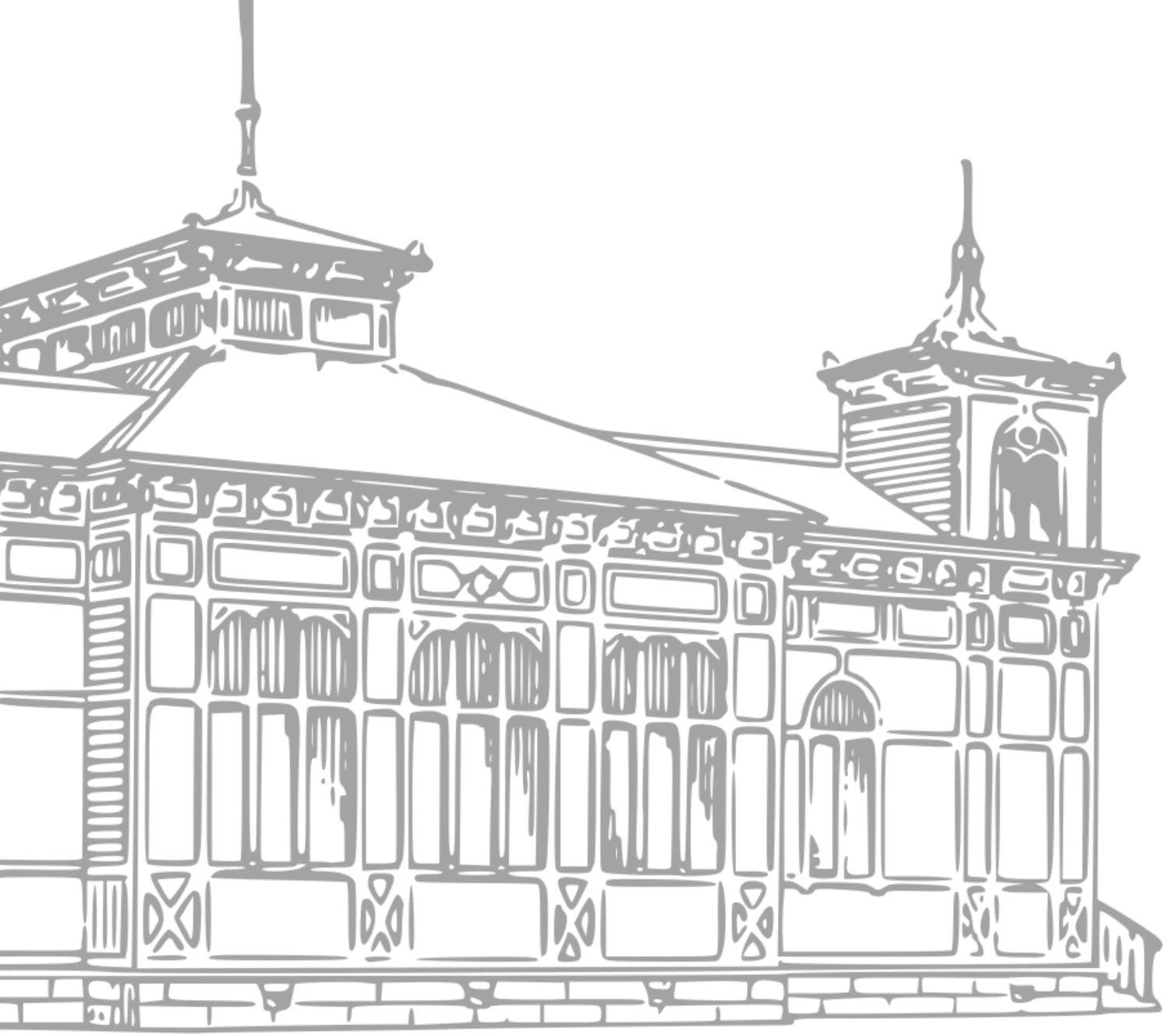
WG11 Issue #	Key Security Vulnerability	WG11 Recommended Mitigation
1	Malicious xApps on Near-RT RIC	Hardware-Rooted Infrastructure Integrity
2	Compromised xApp at Runtime	Continuous Integrity Monitoring (RA)
8	Onboarding Untrusted xApps	Secure Onboarding and Deployment

Objectives & Scope

Primary Objective: Implement a security framework that enhances the resilience of Near-RT RIC against threats from **compromised xApps**.

This objective is achieved through the following specific scope:

1. Infrastructure Trust: We use **Remote Attestation** to verify the **integrity** of the underlying O-RAN infrastructure before deployment. ✓
2. **Workload Identity Attestation**: We implement a mechanism for identity and integrity verification of the xApps themselves. ✓
3. **Automatic Isolation**: Continuous monitoring mechanism for xApps and an isolation pipeline that instantly contains any xApp that violates security protocols.



Progress Overview

Work completed since the project proposal and the **major milestones achieved**

Frameworks & Tools Used

Why ?

Radio Access Network (RAN)

- **srsRAN: O-RAN native support [4]**
- OAI (Open Air Interface): Complex build
- UERANSIM: Limited O-RAN/E2 support

Near-RT RIC

- **O-RAN SC Kubernetes-based RIC: Full O-RAN architecture [5]**
- O-RAN SC Docker-based RIC: Limited lifecycle management
- OAI FlexRIC: Tightly coupled to OAI

Remote Attestation

- **Keylime: Continuous integrity monitoring [8]**
- Intel SGX: Only for specific hardware
- Arm TrustZone: Limited to ARM platforms

Workload Attestation

- **SPIRE/SPIFFE: Standardized workload identity (SVIDs), node and workload attestation [12]**
- WG11 specification report explicitly highlights SPIRE for workload identity

xApp Behavior Monitoring

- **Falco :** eBPF-based kernel-level system call monitoring, customizable rules, real-time alerting **[7]**
- **cAdvisor :** Container resource monitoring (CPU, memory, network, filesystem), Prometheus integration, minimal overhead

Other Tools

- **GPG – GNUPG**
- **Cosign**
- **Docker Engine & Kubeadm**
- **Helm & ChartMuseum**
- **Prometheus**
- **Grafana**

Work Completed and Milestones

Progress Overview 02

Phase 1: Private 5G RAN Testbed with srsRAN and Open5GS

- **srsRAN (CU/DU) + Open5GS 5G Core deployed**
- Virtual radio interface (ZeroMQ) for RF simulation
- End-to-end connectivity validated with srsUE
- Baseline established for xApp testing

Phase 2: Docker-Based Near-RT RIC + xApp Communication

- **O-RAN SC Near-RT RIC deployed via Docker Compose**
- E2AP communication verified with srsRAN CU/DU nodes
- Custom DoS detector xApp deployed as Docker Container
- Attack simulation and & detection workflow validated

Work Completed and Milestones

Progress Overview 02

Phase 3: Kubernetes-Based Near-RT RIC + ONAP SMO

- **kubeadm-based Near-RT RIC deployed in ricplt namespace**
- ONAP-based Service Management and Orchestration (SMO) framework deployed
- Helm-based xApp lifecycle management enabled
- Foundation for zero-trust security integration

Phase 4: Remote Attestation of Nodes (Keylime + vTPM)

- **Keylime framework integrated with vTPM in VMware**
- PCR measurement collection and cryptographic quoting
- Continuous integrity monitoring of RIC host
- Hardware-rooted trust chain established

Work Completed and Milestones

Progress Overview 02

Phase 5: Secure xApp Onboarding + Workload Attestation

- End-to-end secure onboarding pipeline implemented
- **GPG and Cosign** signature verification for xApp descriptors and container images
- Integration of **SPIRE/SPIFFE for workload attestation of xApps**
- Dynamic X.509 SVID issuance for mTLS authentication with RIC appmgr

Phase 6: Evaluation of Runtime Monitoring Tools for xApps

- Two-layer monitoring for security + performance metrics
- **Falco**: Detects shell injection, file access, process spawning and network anomalies
- **cAdvisor: Real-time CPU/memory/network metrics** with threshold-based alerting
- Foundation for automated containment pipeline

Initial Testbed Setup - Testing

Open 5GS 5G Core → Docker-based O-RAN SC Near-RT RIC → srsCU → srsDU → srsUE → Pinging the 5G Core from the User Equipment → Traffic Monitoring in the srsDU

```
janindu@janindudewmith:~/Desktop/FYP/srsRAN/srsRAN_Project/configs$ srsdu -c du.yml

---- srsRAN DU (commit d5fa4f0ecb) ----

Lower PHY in executor blocking mode.
Available radio types: zmq.
Cell pci=1, bw=20 MHz, 1T1R, dl_arfcn=368500 (n3), dl_freq=1842.5 MHz, dl_ssb_arfcn=368410, ul_freq=1747.5 MHz

F1-C: Connection to CU-CP on 127.0.10.1:38472 completed
E2AP: Connection to Near-RT-RIC on 10.0.2.10:36421 completed
==== DU started ===
Type <h> to view help
t
```

		DL						UL								
pci	rnti	cqi	ri	mcs	brate	ok	nok	(%)	dl_bs	pusch	rsrp	ri	mcs	brate	ok	nok	(%)	bsr	ta	phr
1	4601	15	1.0	27	53k	40	0	0%	0	42.8	ovl	1	28	274k	63	0	0%	0	520n	n/a
1	4601	15	1.0	27	43k	32	0	0%	0	42.7	ovl	1	28	279k	64	0	0%	102	520n	n/a
1	4601	15	1.0	27	35k	27	0	0%	0	42.8	ovl	1	28	257k	59	0	0%	102	520n	n/a
1	4601	15	1.0	27	37k	28	0	0%	0	42.8	ovl	1	28	248k	57	0	0%	0	520n	n/a
1	4601	15	1.0	27	40k	31	0	0%	0	42.8	ovl	1	28	270k	62	0	0%	10	520n	n/a
1	4601	15	1.0	27	37k	29	0	0%	0	42.8	ovl	1	28	283k	65	0	0%	0	520n	n/a
1	4601	15	1.0	27	32k	25	0	0%	0	42.8	ovl	1	28	218k	50	0	0%	102	520n	n/a

Figure 02: Monitoring the simulated traffic from the srsUE, in the DU logs

Detailed Progress

Technical implementation, experimental validation and performance analysis of the security framework

Remote Attestation Architecture

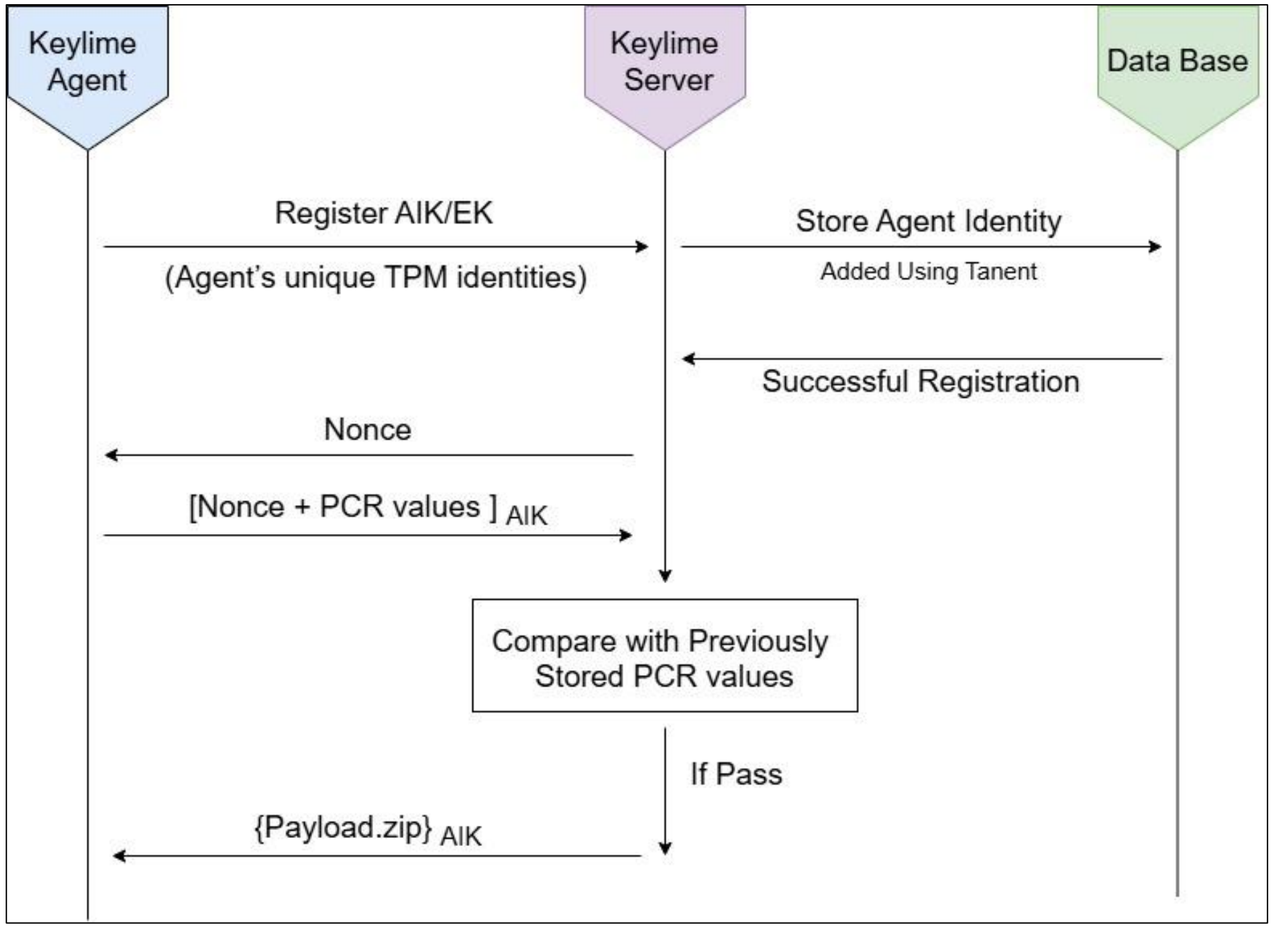
What is Remote Attestation?

- It is a security process where a Server verifies the **health** and **Integrity** of a remote node before allowing it to join the network
- The goal is to ensure the underlying O-RAN infrastructure is clean and untampered with prior to any deployment **[10]**

• As our trust Broker we use **Keylime** because it act as an automated framework to manage security checks at scale. Unlike traditional security tools that only check a system at boot up, Keylime performs Continuous Attestation which is very important for near real time communication **[8]**

• **Visual Monitoring** : We deployed a python interface to provide a real-time interface to show the agent's **status** in real time

Remote Attestation Communication Flow



- Our system uses a 2s attestation loop with fresh nonces to **block replay attacks** and ensure real-time infrastructure monitoring
- We transitioned to **IMA** driven automation, utilizing a "Golden Standard" Allowlist to instantly detect unauthorized changes
- Secure payloads are only released and decrypted once the node successfully proves its **hardware integrity**

Figure 03: Remote Attestation Communication Flow Between Agent and Server

Demo

Remote Attestation Experimental Validation

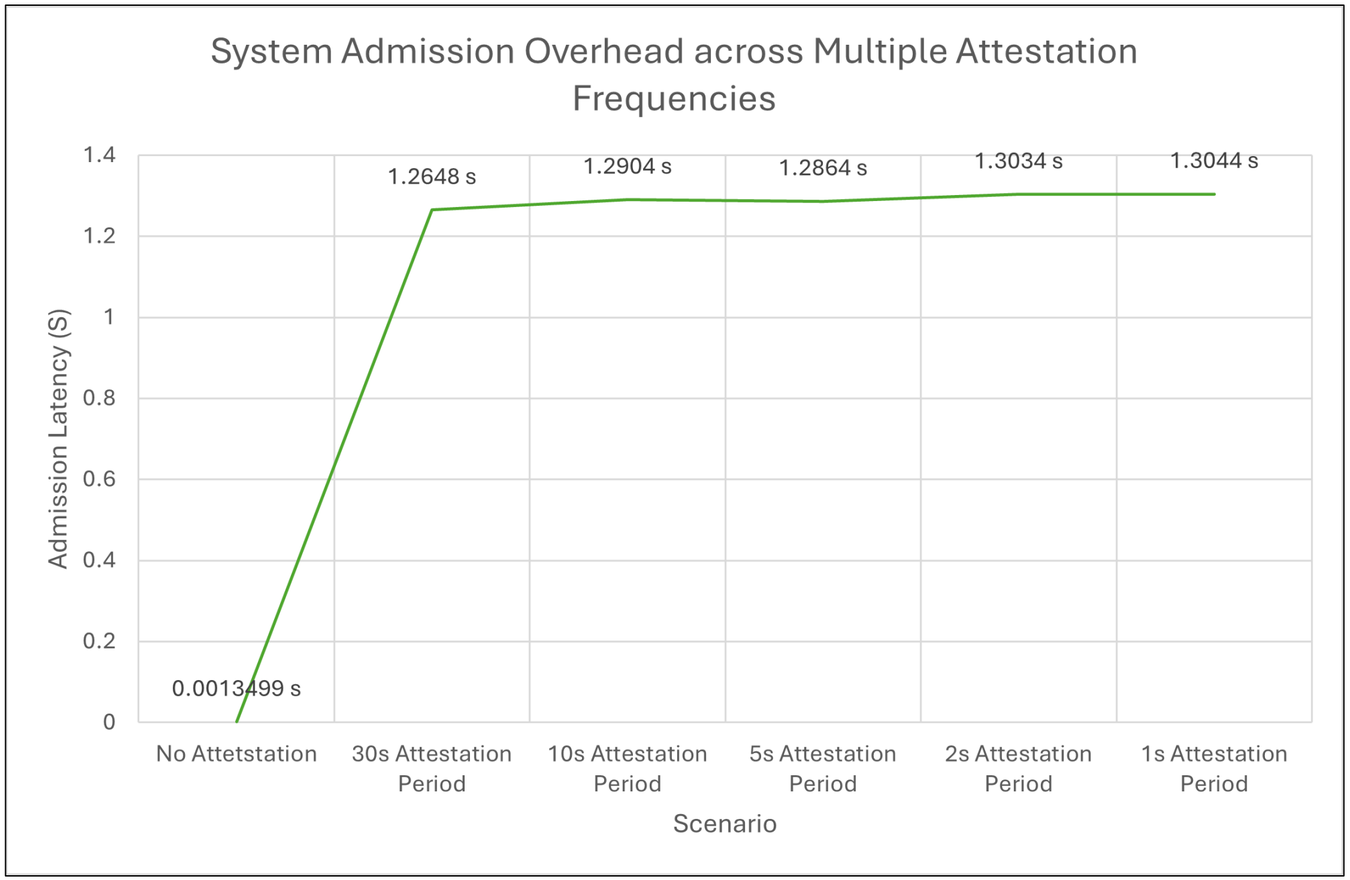


Figure 04: Remote Attestation Admission Time Variation to Different Scenarios

- We measured the time from the initial Agent request to **full payload delivery**, comparing a baseline TCP handshake to a secured attestation process

- The results demonstrated that the **attestation time** is **nearly independent** of the deployment time or the frequency of the attestation intervals

Kubernetes RIC Configurations


```
deshanric@deshanric-virtual-machine:~$ kubectl get pods -n ricplt
NAME                                                READY   STATUS    RESTARTS      AGE
deployment-ricplt-a1mediator-64bff884f9-854sr       1/1     Running   1 (22m ago)   23m
deployment-ricplt-alarmmanager-cbb797dfd-r9zvz      1/1     Running   0             23m
deployment-ricplt-appmgr-779896945f-xbqfb           1/1     Running   0             23m
deployment-ricplt-e2mgr-6c8b9f97-c299s              1/1     Running   3 (22m ago)   23m
deployment-ricplt-e2term-alpha-587b8bccb6-4ndjs     1/1     Running   0             23m
deployment-ricplt-o1mediator-59d669865-ndgx9        1/1     Running   0             23m
deployment-ricplt-rtmgr-7848fb5964-bgbr8            1/1     Running   1 (22m ago)   23m
deployment-ricplt-submgr-56684798c-fpftt            1/1     Running   0             23m
deployment-ricplt-vespamgr-8d49dd658-xq9tb          1/1     Running   0             23m
deployment-ricplt-xapp-onboarder-78c4698978-jlwlh   2/2     Running   0             23m
r4-infrastructure-kong-5f599d77fc-8nqk6             2/2     Running   0             23m
r4-infrastructure-prometheus-alertmanager-6755c5b76d-blcnz   2/2     Running   0             23m
r4-infrastructure-prometheus-server-7f99b65b5c-mdvwv         1/1     Running   0             23m
statefulset-ricplt-dbaas-server-0                   1/1     Running   0             23m
deshanric@deshanric-virtual-machine:~$ kubectl get pods -n spire-system
NAME                                                READY   STATUS    RESTARTS        AGE
spiffe-csi-driver-s6jf2                             2/2     Running   21 (26m ago)    3d
spire-agent-qh5kq                                   1/1     Running   2 (45h ago)     2d23h
spire-server-0                                      2/2     Running   48 (26m ago)    4d5h
spire-spiffe-oidc-discovery-provider-68d6d99b79-tzbw4   2/2     Running   23 (19m ago)    4d5h
```

Figure 05: RIC Platform and SPIRE System

SMO : Service Management Orchestration

- ONAP SMO : Full Orchestration (Heavy - 2022)
- Implemented only required components relevant to our project scope

Secure Onboarding with Workload Attestation (WA) [6], [9]

Virtual Machines Component Architecture

Detailed Progress

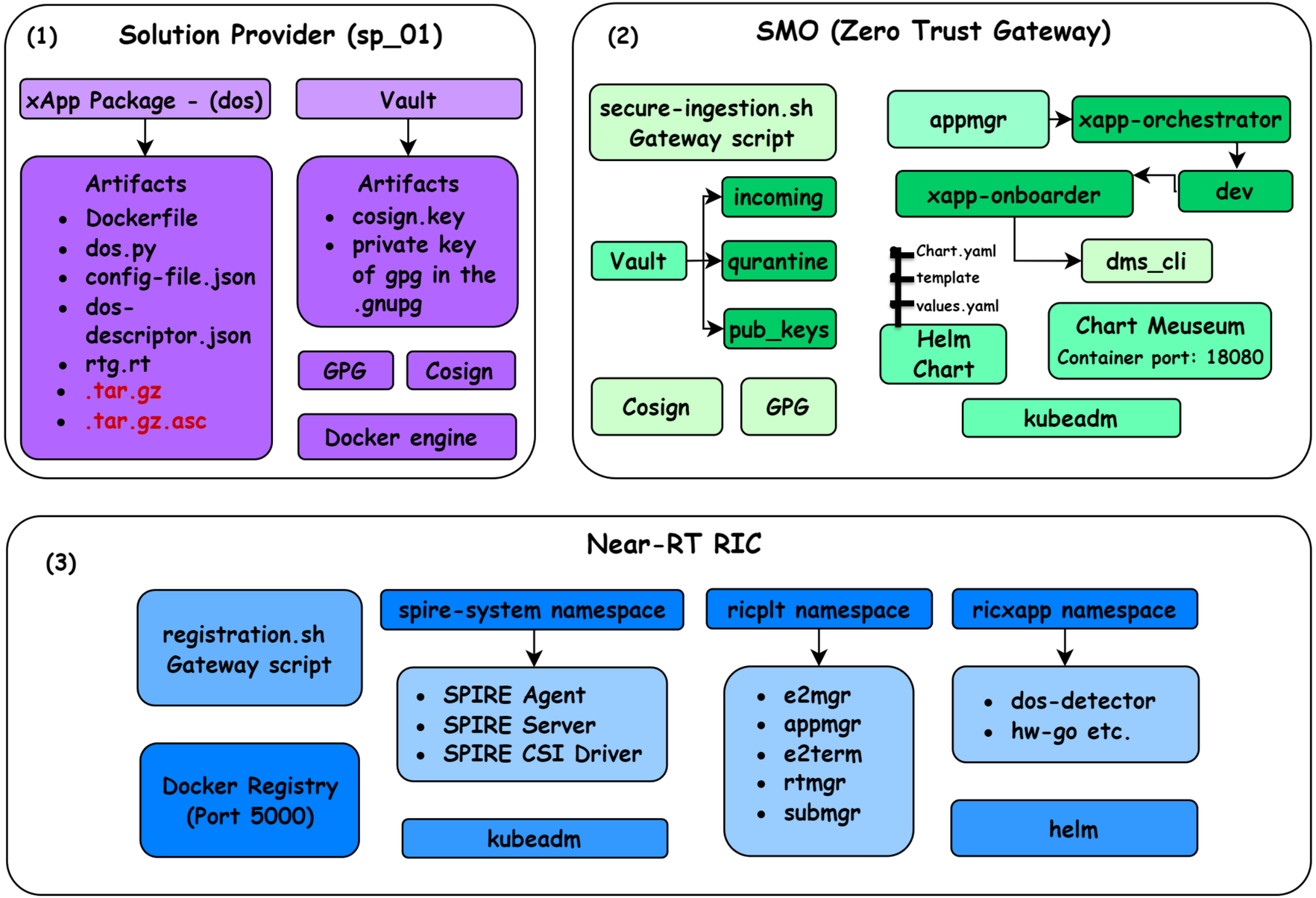


Figure 06: Detailed end to end secure on boarding with workload Attestation components

Solution Provider Process

Detailed Progress

03

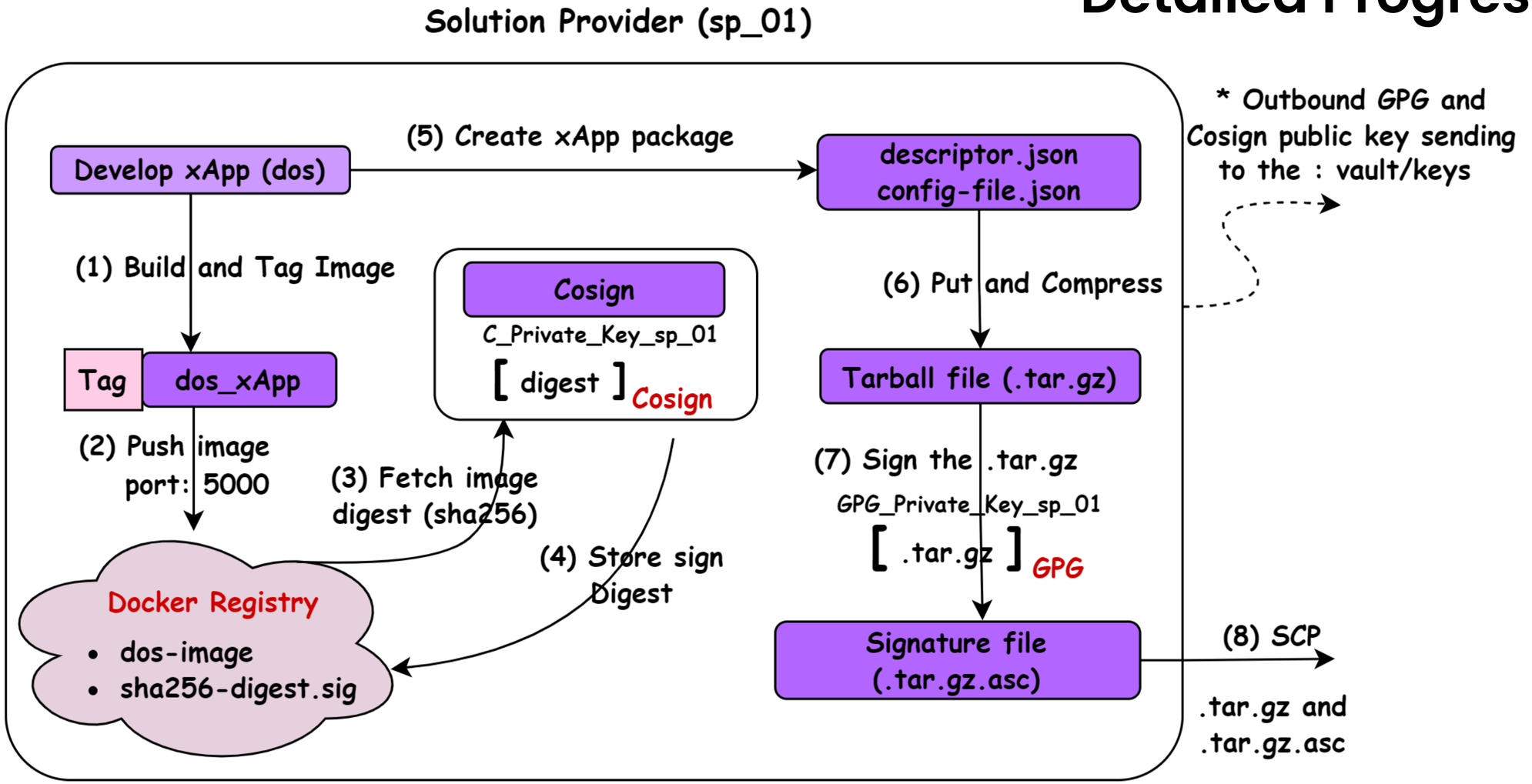
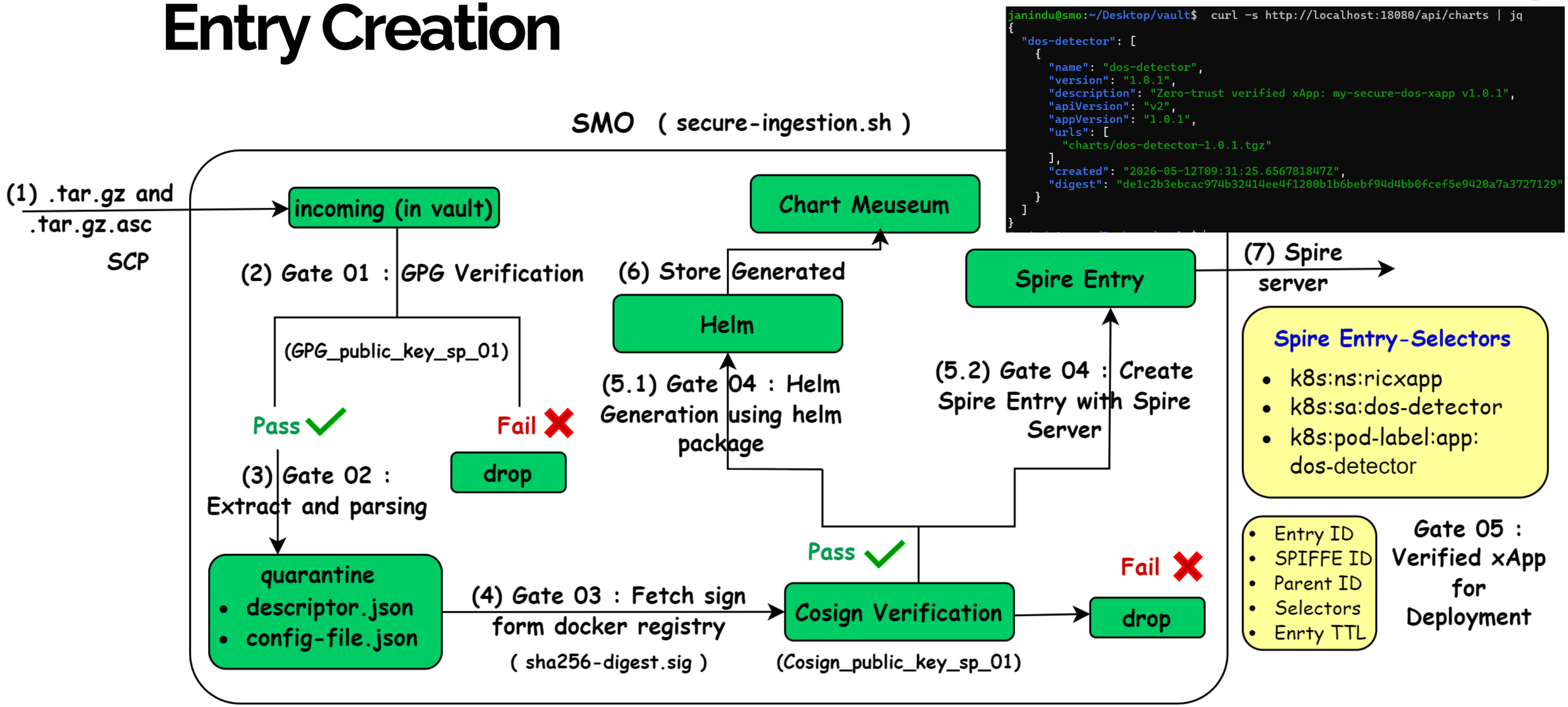


Figure 07: Solution Provider End to End Process

```
solutionprovider@ubuntu:~/Desktop/dos_xapp_package$ scp ~/Desktop/dos_xapp_package/dos-detector-d
escriptor.tar.gz* janindu@192.168.8.142:~/Desktop/vault/incoming/
janindu@192.168.8.142's password:
dos-detector-descriptor.tar.gz                100%  662   132.7KB/s   00:00
dos-detector-descriptor.tar.gz.asc            100%  862   155.7KB/s   00:00
```

SMO Process & SPIRE Entry Creation




```
janindu@smo:~/Desktop/vault$  curl -s http://localhost:18080/api/charts | jq
{
  "dos-detector": [
    {
      "name": "dos-detector",
      "version": "1.0.1",
      "description": "Zero-trust verified xApp: my-secure-dos-xapp v1.0.1",
      "apiVersion": "v2",
      "appVersion": "1.0.1",
      "urls": [
        "charts/dos-detector-1.0.1.tgz"
      ],
      "created": "2026-05-12T09:31:25.6567818477",
      "digest": "de1c2b3ebcac974b32414ee4f1200b1b6bebf94d4bb0fcef5e9420a7a3727129"
    }
  ]
}
```

Figure 08: SMO VM End to End Process

Results : Trusted xApp Onboarding ✓

Detailed Progress

03

```
================================================================================
                ZERO-TRUST O-RAN INGESTION PIPELINE DASHBOARD
================================================================================
  ▸ Provider ID       | sp1
  ▸ Payload           | dos-detector-descriptor.tar.gz
  ▸ Target RIC        | 192.168.8.184

================================================================================
       [ GATE 1 ] ORIGIN SIGNATURE VERIFICATION (GPG)
--------------------------------------------------------------------------------
  ↳ Public Key        | keys/sp1_gpg.pub
  ✔ Status            | [ PASSED ] Origin verified. Payload is authentic.

================================================================================
       [ GATE 2 ] PAYLOAD EXTRACTION & DESCRIPTOR PARSING
--------------------------------------------------------------------------------
  ↳ Workspace         | quarantine/ directory populated
  ↳ xApp Name         | my-secure-dos-xapp
  ↳ Chart Name        | dos-detector
  ↳ Target Image      | 192.168.8.184:5000/dos-detector:1.0.1
  ✔ Status            | [ PASSED ] Architectural descriptors parsed.

================================================================================
       [ GATE 3 ] EDGE REGISTRY IMAGE INTEGRITY (COSIGN)
--------------------------------------------------------------------------------
  ↳ Registry          | 192.168.8.184:5000
  ↳ Cosign Key        | keys/sp1_cosign.pub
  ✔ Status            | [ PASSED ] Image cryptographically verified on Edge.
```

```
================================================================================
       [ GATE 4 ] PARALLEL ORCHESTRATION (HELM VAULT + SPIRE)
--------------------------------------------------------------------------------
  ↳ Agent ID          | spiffe://oran.fyp.local/spire/agent/k8s_psat/cluster/kub
                      | ernetes/node/deshanric-virtual-machine
  ✔ Helm Vault        | [ SEALED ] Chart 'dos-detector' locked (HTTP 201)
  ✔ SPIRE Identity    | [ REGISTERED ] SPIFFE ID issued for 'dos-detector'

================================================================================
       [ GATE 5 ] VERIFIED XAPP READY FOR RIC DEPLOYMENT
--------------------------------------------------------------------------------
  ↳ Chart Name        | dos-detector
  ↳ Version           | 1.0.1
  ↳ Helm Repo         | http://192.168.8.142:18080
  ↳ Namespace         | ricxapp
  ↳ RIC VM            | 192.168.8.184

================================================================================
       [ FINALIZATION ] PIPELINE CLEANUP & VERIFICATION
--------------------------------------------------------------------------------
  ↳ Task              | Fetching verified SPIFFE Identity from Server...
  ↳ Entry ID          | 18c7eec1-3c67-406f-b71b-4d05c51faaa6
  ↳ SPIFFE ID         | spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector
  ↳ Parent ID         | spiffe://oran.fyp.local/spire/agent/k8s_psat/cluster/kub
                      | ernetes/node/deshanric-virtual-machine
  ↳ Selectors         | k8s:ns:ricxapp,k8s:pod-label:app:dos-detector k8s:sa:dos
                      | -detector
  ↳ SVID TTL          | 3600s
--------------------------------------------------------------------------------
                [ ✔ SUCCESS ] xApp Trusted & Ready to Deploy
================================================================================
```

Figure 09: SMO End to End Process Results

Demo

Results : Threat Model (Integrity)

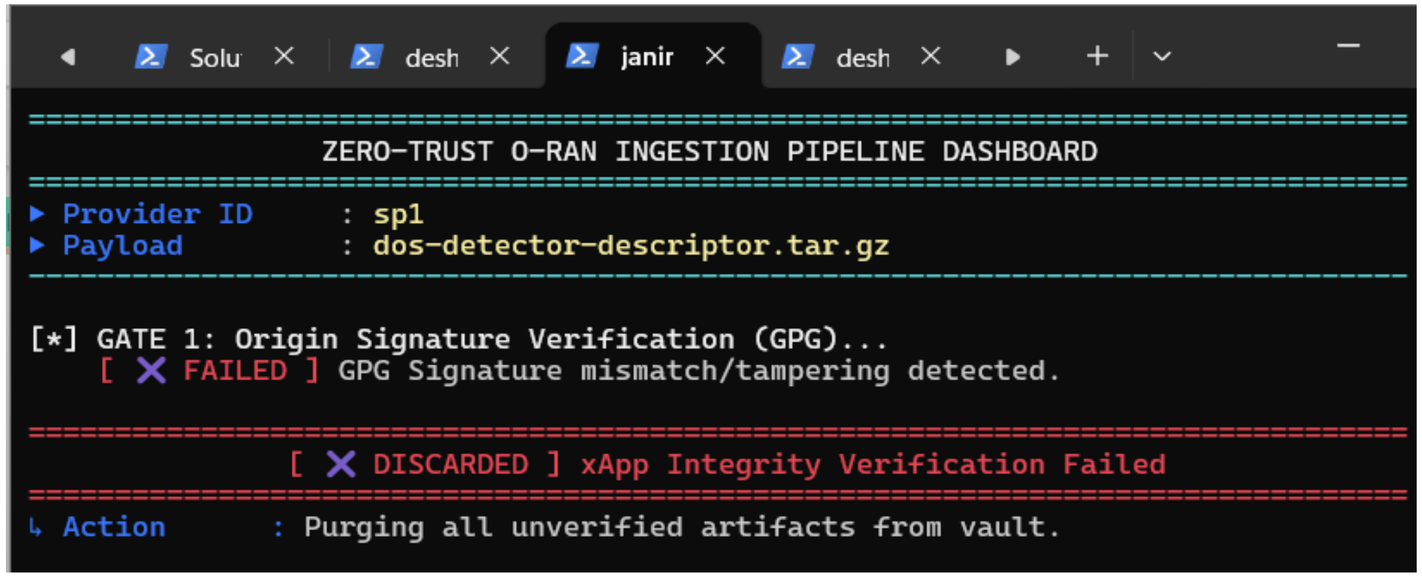
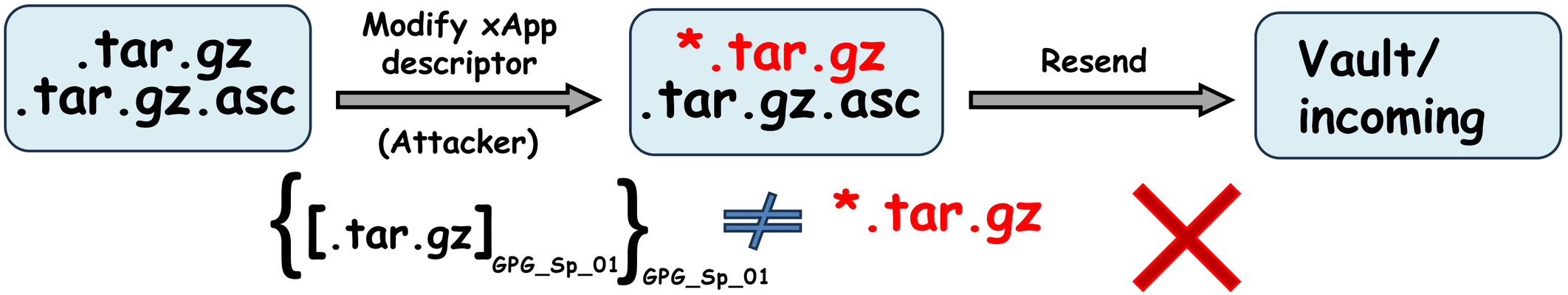
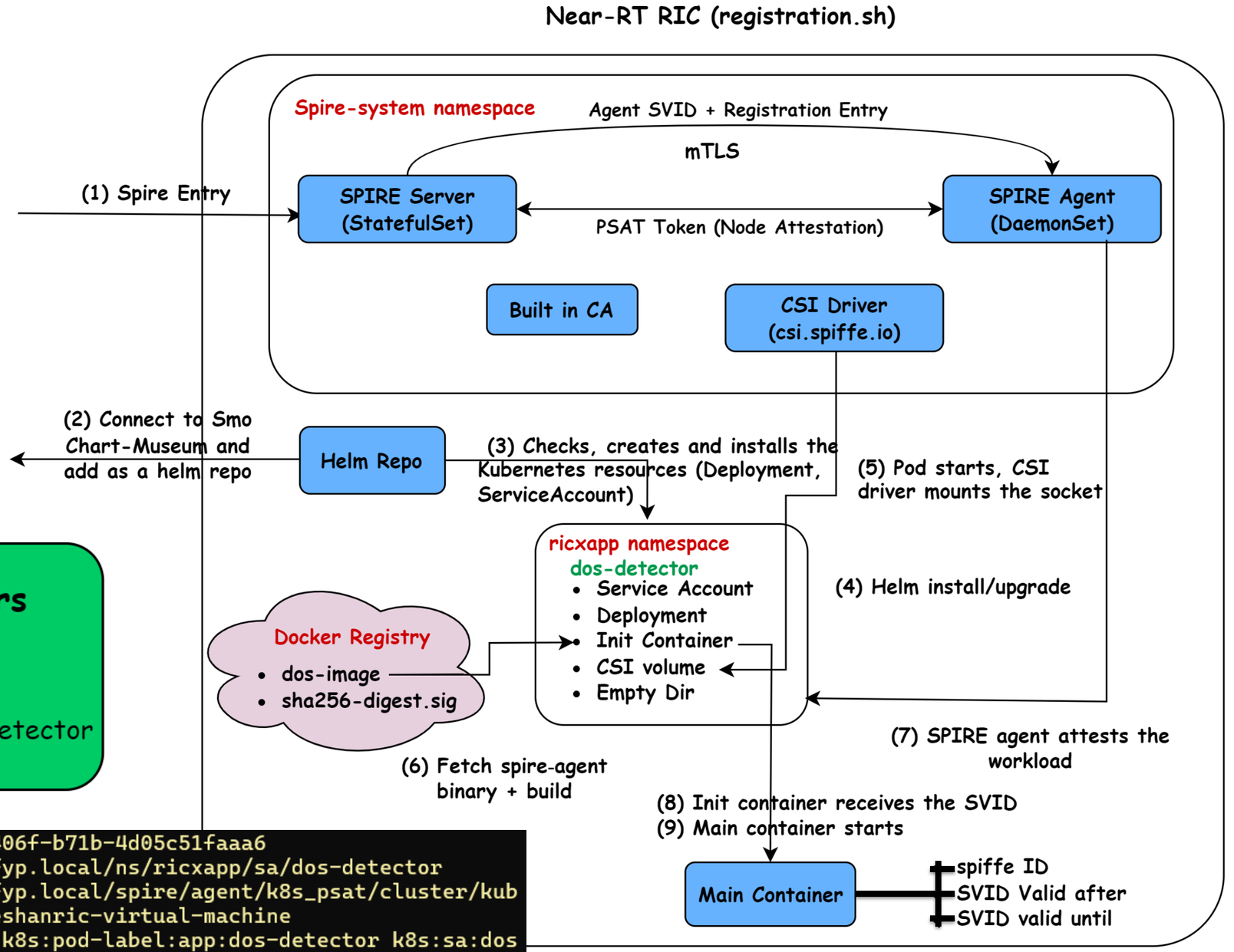


Figure 10: Code Integrity Testing Results

$$[.tar.gz]_{GPG\_Sp\_01} = .tar.gz.asc$$



Workload Attestation [6]



Spire Entry Selectors

- k8s:ns:ricxapp
- k8s:sa:dos-detector
- k8s:pod-label:app:dos-detector

```
Entry ID         | 18c7eec1-3c67-406f-b71b-4d05c51faaa6
SPIFFE ID        | spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector
Parent ID        | spiffe://oran.fyp.local/spire/agent/k8s_psat/cluster/kub
                   ernetes/node/deshanric-virtual-machine
Selectors        | k8s:ns:ricxapp,k8s:pod-label:app:dos-detector k8s:sa:dos
                   -detector
```

Figure 11: Near RT RIC VM End to End Process

Results : Deployment and Workload Attestation


```
================================================================================
                NEAR-RT RIC: xAPP DEPLOYMENT & ATTESTATION DASHBOARD
================================================================================
  ▸ xApp Name          | dos-detector
  ▸ Version            | 1.0.1
  ▸ Repo               | http://192.168.8.142:18080

================================================================================
  [ STEP 1 ] CONNECTING TO SMO CHARTMUSEUM & SERVICEACCOUNT VERIFICATION
--------------------------------------------------------------------------------
  ↳  Repo URL          | http://192.168.8.142:18080
  ✔  Status            | [ PASSED ] Successfully synced with SMO repository.
  ↳  ServiceAccount    | dos-detector
  ✔  Status            | [ PASSED ] ServiceAccount already present in namespac
                       | e 'ricxapp'.
  ✔  Status            | [ PASSED ] ServiceAccount is already owned by Helm.

================================================================================
  [ STEP 2 ] INSTANTIATING XAPP VIA HELM & DEPLOYMENT ROLLOUT
--------------------------------------------------------------------------------
  ↳  Helm              | [ UPGRADED ] Existing xApp updated to v1.0.1.
  ↳  Active Pod        | dos-detector-78bdcfc94-2cxgw
  ↳  ServiceAccount    | dos-detector
  ✔  SA Match          | [ PASSED ] Container is using the correct ServiceAcco
                       | unt.
```

```
deshanric@deshanric-virtual-machine:~/Desktop/registration$ kubectl get
pods -n ricxapp
NAME                                READY   STATUS    RESTARTS        AGE
dos-detector-78bdcfc94-2cxgw        1/1     Running   0               13m
ricxapp-hw-go-55576c899f-ttx8j      1/1     Running   5 (2d6h ago)    15d
rogue-xapp                          1/1     Running   3 (2d6h ago)    12d
secure-fyp-xapp-5bbb86ddc9-9g8fv    2/2     Running   6 (2d6h ago)    12d
```

```
================================================================================
  [ STEP 3 ] ZERO-TRUST WORKLOAD ATTESTATION
--------------------------------------------------------------------------------
  ↳  Layer A           | SPIRE server entry check...
  ✔  Status            | [ PASS ] Registration entry found on SPIRE server (2
                       | total).
  ↳  SPIFFE ID         | spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector
  ↳  Layer B           | SVID issuance confirmation (init container logs)...
  ✔  Status            | [ PASS ] Init container log confirms SVID received:

  Waiting for SPIRE socket...
  Socket ready. Fetching SVID...
  wget: note: TLS certificate validation not implemented
  Received 1 svid after 21.997671ms

  SPIFFE ID:                  spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector
  Hint:                       default
  SVID Valid After:           2026-05-12 09:31:56 +0000 UTC
  SVID Valid Until:           2026-05-12 13:32:06 +0000 UTC
  Intermediate #1 Valid After: 2026-05-12 09:26:01 +0000 UTC
  Intermediate #1 Valid Until: 2026-05-13 09:26:01 +0000 UTC
  CA #1 Valid After:          2026-04-23 09:07:52 +0000 UTC
  CA #1 Valid Until:          2036-04-20 09:07:52 +0000 UTC

  Writing SVID #0 to file /svid-data/svid.0.pem.
  Writing key #0 to file /svid-data/svid.0.key.
  Writing bundle #0 to file /svid-data/bundle.0.pem.
  SVID fetched successfully.


================================================================================
  [ SUMMARY ] ATTESTATION RESULTS
--------------------------------------------------------------------------------
  ✔  Result            | [ ATTESTATION COMPLETE ] Both layers passed. xApp ide
                       | ntity proven.

================================================================================
                [ ✔ SUCCESS ] xApp is Active and Cryptographically Secured
================================================================================
```

Figure 12: Near RT RIC VM Process Results and ricxapp Namespace

Demo

xApp Behavior Monitoring

Falco

- Continuously tests every system call against defined security policies
- Falco GUI for monitoring malicious Events and alerts
- Falco has default set of rules
- We can customize our own rules **[7]**

Results :

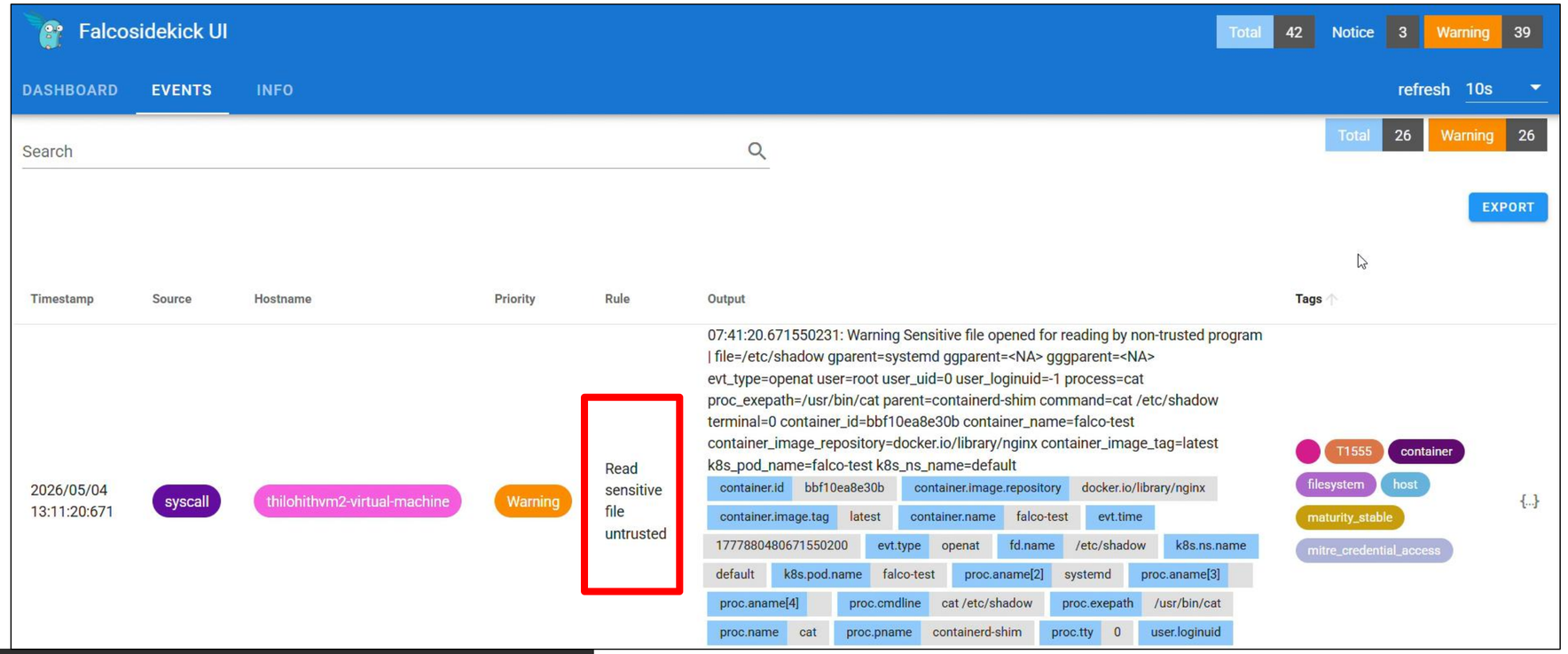


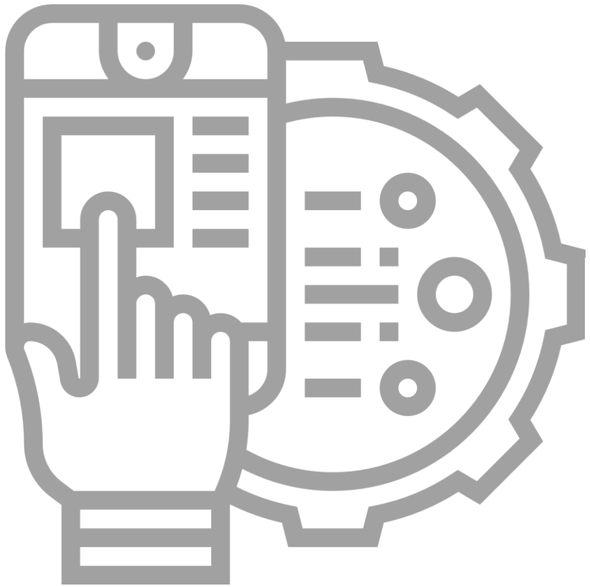
Figure 13: Falco image Dashboard

```
thilohithvm2@thilohithvm2-v
thilohithvm2@thilohithvm2-vi
NAME          READY   STATUS     RESTARTS   AGE   IP            NODE                          NOMINATED NODE   READINESS GATES
falco-tv9xh   0/2     Init:0/1   0          33s   10.244.0.36   thilohithvm2-virtual-machine   <none>           <none>
thilohithvm2@thilohithvm2-virtual-machine:~$ kubectl get pods -n falco -w
NAME          READY   STATUS    RESTARTS   AGE
falco-tv9xh   2/2     Running   0          87s
thilohithvm2@thilohithvm2-virtual-machine:~$ kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
thilohithvm2@thilohithvm2-virtual-machine:~$ kubectl exec -it $(kubectl get pods --selector=app=nginx -o name) -- cat /etc/shadow
error: unable to upgrade connection: container not found ("nginx")
thilohithvm2@thilohithvm2-virtual-machine:~$ kubectl exec -it $(kubectl get pods --selector=app=nginx -o name) -- cat /etc/shadow
root:*:20564:0:99999:7:::
daemon:*:20564:0:99999:7:::
bin:*:20564:0:99999:7:::
sys:*:20564:0:99999:7:::
sync:*:20564:0:99999:7:::
games:*:20564:0:99999:7:::
man:*:20564:0:99999:7:::
lp:*:20564:0:99999:7:::
mail:*:20564:0:99999:7:::
news:*:20564:0:99999:7:::
uucp:*:20564:0:99999:7:::
proxy:*:20564:0:99999:7:::
www-data:*:20564:0:99999:7:::
backup:*:20564:0:99999:7:::
list:*:20564:0:99999:7:::
irc:*:20564:0:99999:7:::
_apt:*:20564:0:99999:7:::
nobody:*:20564:0:99999:7:::
nginx:!:20565:::::
```

Figure 14: Falco attack simulation

Demo

Resource Utilization monitoring



- Achieved through cAdvisor, Prometheus, and Grafana.
- We monitor CPU usage, memory usage, and Network.
- Receive alerts by setting threshold of usage.

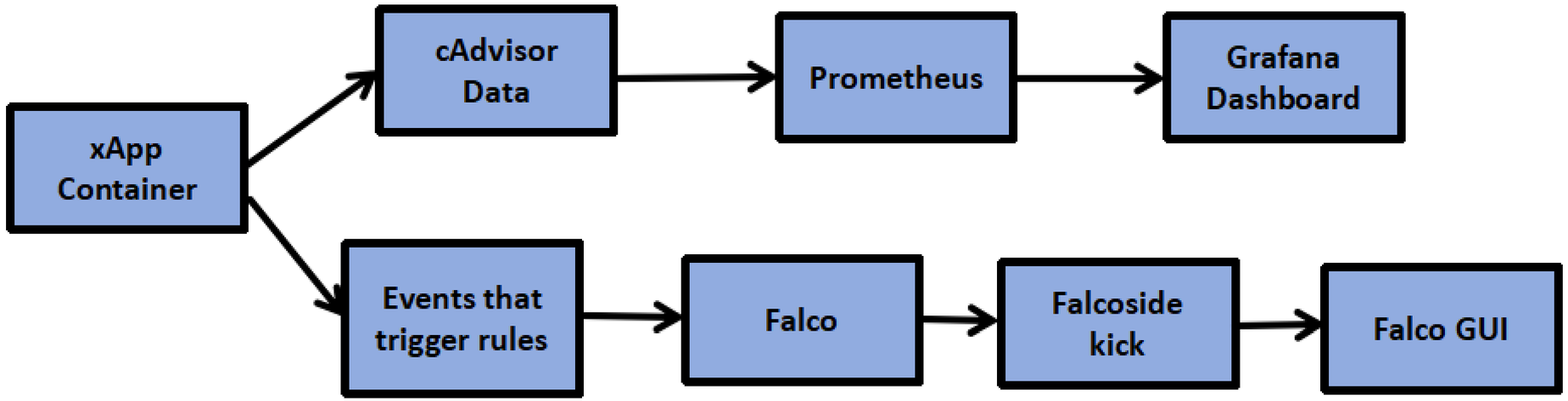


Figure 15: Process flow of xApp monitoring system

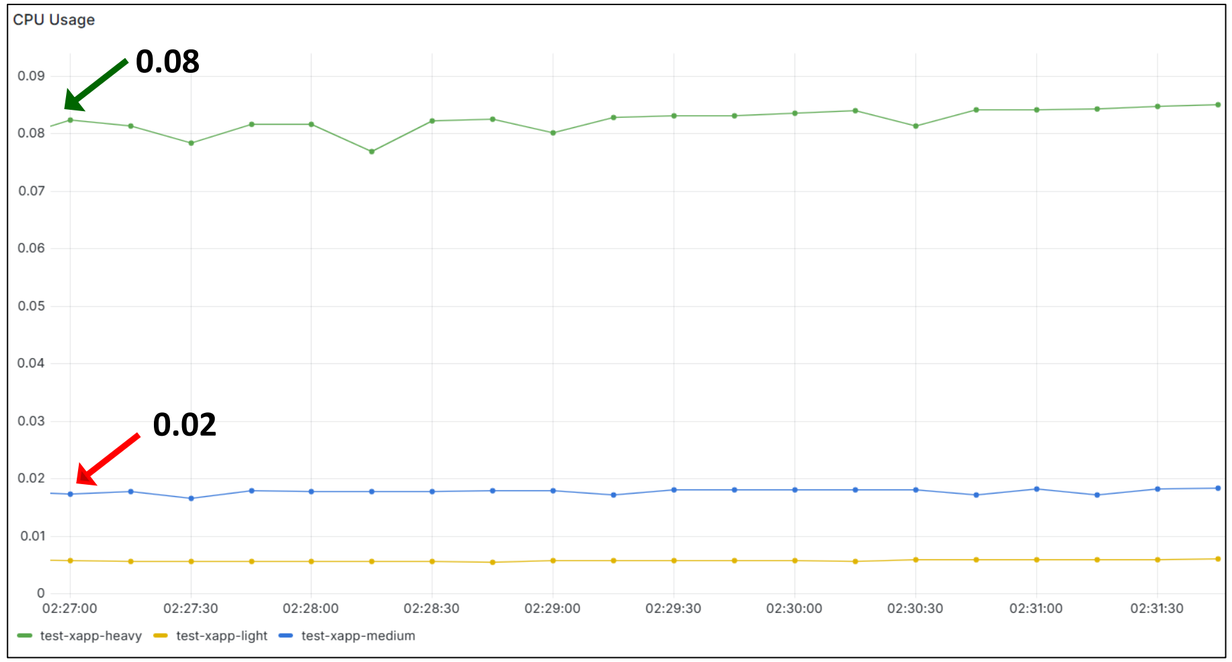


Figure 16: Normal CPU usage of three different xApps

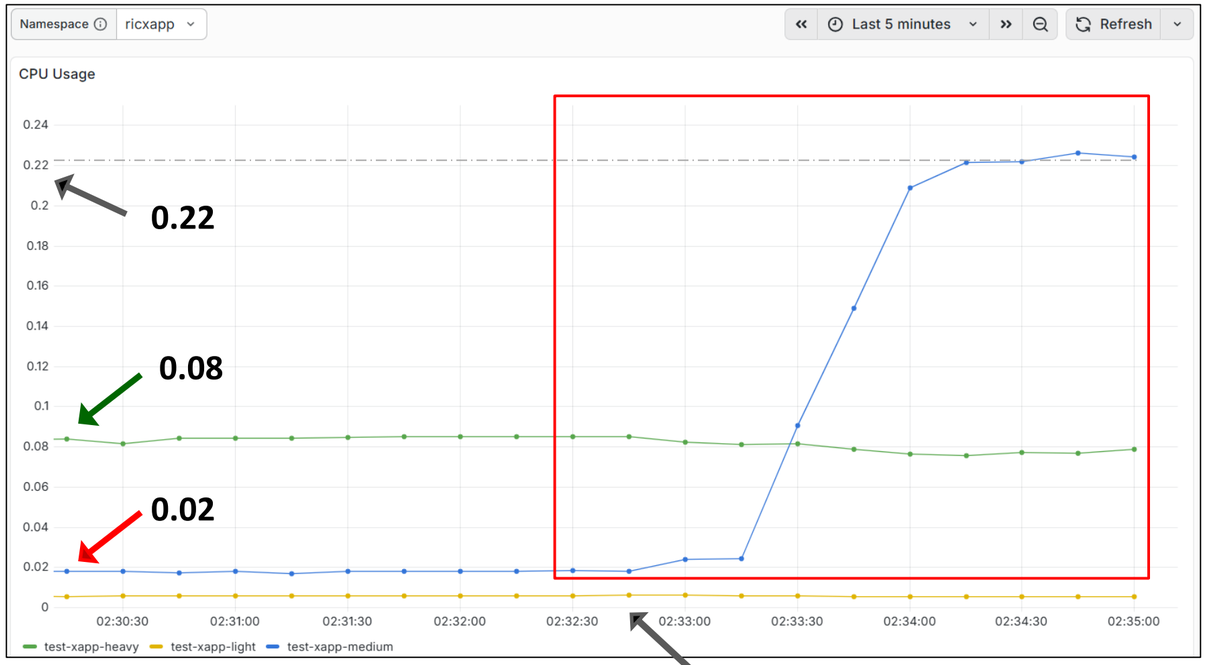
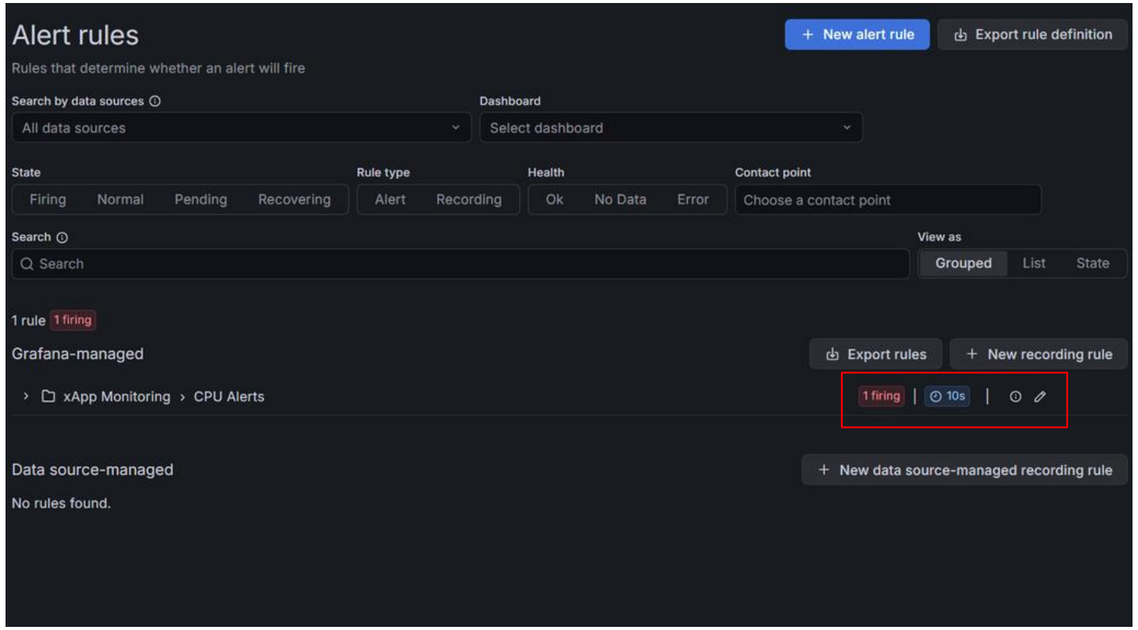


Figure 17: CPU usage after giving heavy load to the **blue colour medium** one

Results :

- **Green** - Test xApp Heavy
- **Blue** - Test xApp Medium*
- **Yellow** - Test xApp Light



Time: 02 : 32 : 45

Figure 18: Excess CPU usage alert dashboard

Challenges & Solutions 04

Hardware-Rooted Trust without Physical TPM

- **Problem**: Remote attestation **requires TPM 2.0**, but we are using VMs so we use **vTPM**
- **Solution**: Configured virtual TPM (vTPM) in VMware
- Validated PCR measurement collection works **identical to physical TPM**

Init Container Internet Dependency in Workload Attestation

- **Problem**: The SPIRE agent download its binary **from GitHub** at pod startup, which **failed when the RIC VM had no internet access (After reboot)**
- **Solution**: Built a custom **pre-packaged spire-fetcher image** that already contains SPIRE agent binary

Challenges & Solutions 04

SPIRE Selector Mismatch

- **Problem**: Registration entry used **common selectors** that the **SPIRE attestor never recognized** as matching pod's metadata
- **Solution**: **Redesigned the entry with three separate, exact selectors**: namespace, service account, pod label. This guarantees that only a pod meeting all three criteria can obtain an SVID

Spire Entry Selectors

- k8s:ns:ricxapp
- k8s:sa:dos-detector
- k8s:pod-label:app:dos-detector

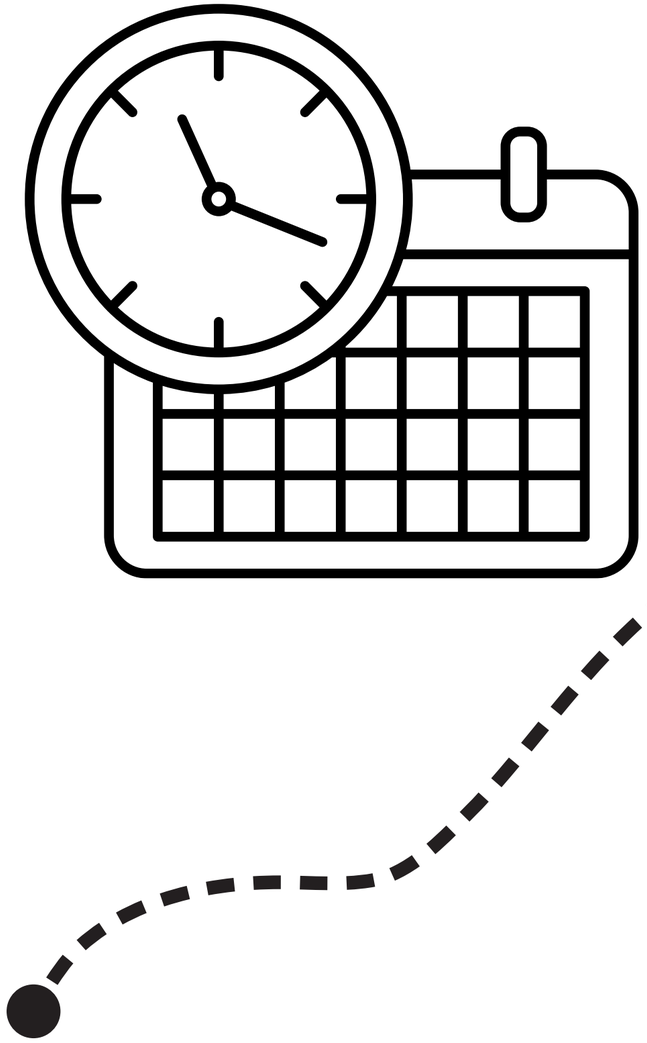
Falco UI not accessible

- **Problem**: After configuring Falco using Helm charts, **Falco UI was not accessible** from the browser, while trying to access
- **Solution**: Changed the service type to Node port which exposes the **service on a static port** to all nodes in the cluster

Future Work

Timeline

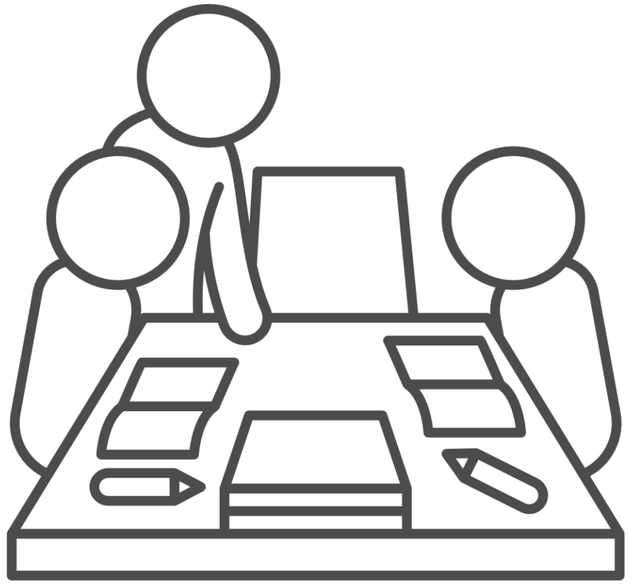
- **May**
  - Customize rules in Falco
  - Integrating Falco and cAdvisor for actual xApp monitoring
- **June**
  - Attack simulation and **quarantine**
  - Isolation of compromised xApp.
- **July**
  - Report preparation
  - End to end testing of solution
- **August**
  - Final Demo and presentation



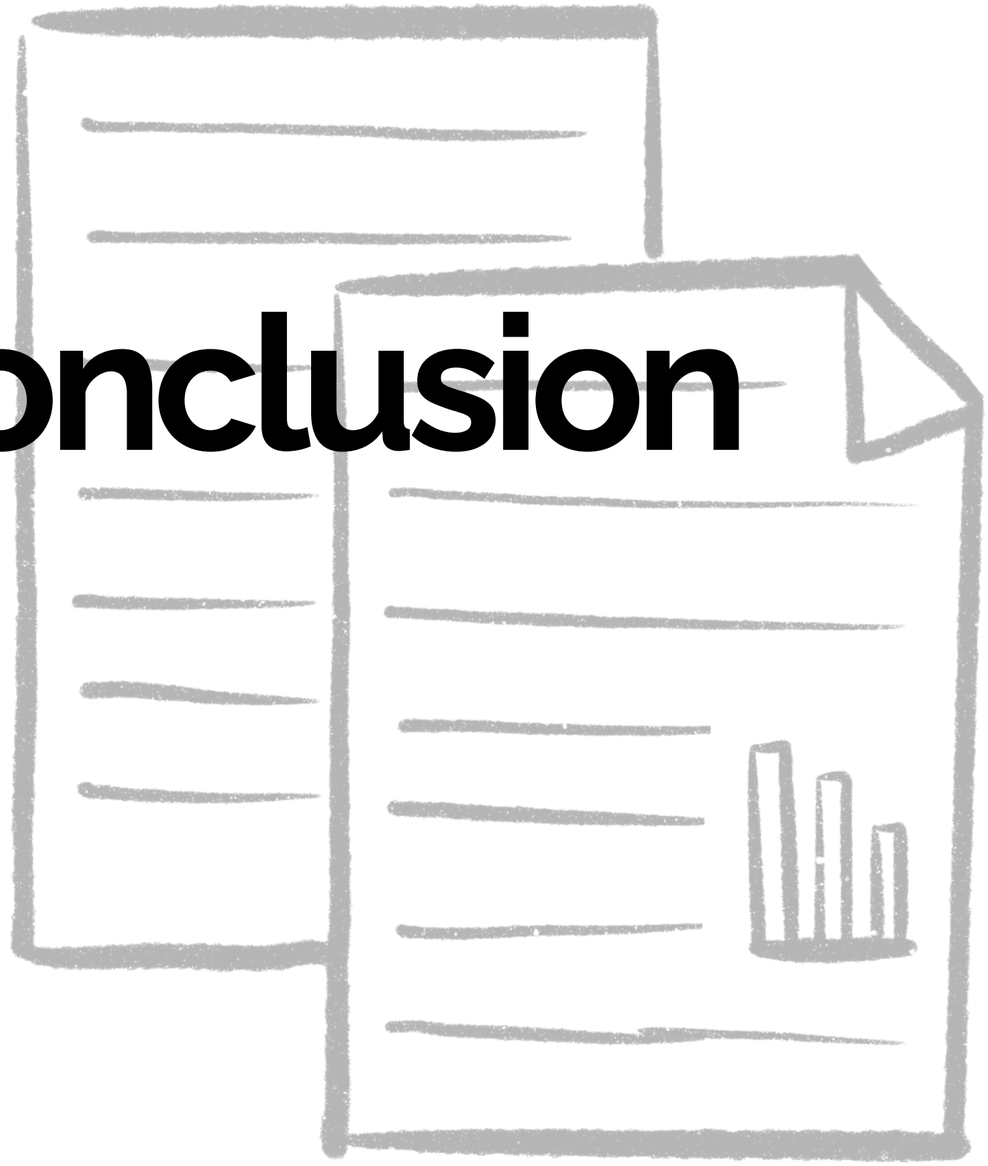
Next Steps & Conclusion

05

- Integrating Falco and cAdvisor to actual **xApp monitoring**
- Quarantine and isolation of compromised xApps after successful detection from continuous monitoring
- **Inter connecting all subsystems**
- Simulating threat models using **different attack types**
- Integrating different dashboards to a common GUI



Conclusion



Conclusion 05

- Deployed **end-to-end Open RAN testbed using srsRAN, Open5GS, Near-RT RIC, and customized SMO**.
- Implemented a secure xApp onboarding framework using:
  - Remote Attestation with **Keylime** and TPM/vTPM
  - **Workload Identity Attestation using SPIRE/SPIFFE**
  - Cryptographic verification using **Cosign** and **GPG**
- Demonstrated runtime **monitoring** of pods using:
  - Falco
  - Prometheus/cAdvisor
  - Grafana dashboards

References

[1]. M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," IEEE Communications Surveys & Tutorials, vol. 25, no. 2, pp. 1376 1411, 2023

[2]. O-RAN Working Group 1, "O-RAN Architecture Description," O-RAN AL LIANCE e.V., Tech. Rep. TR.0-R004-v15.00, Oct. 2025

[3]. O-RANWorkingGroup3, "Near-RTRICArchitecture," O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v07.00, Feb. 2025

[4]. "srsRAN Documentation," https://docs.srsran.com/projects/project/en/ latest/index.html, srsRAN Project

[5]. "O-RAN Software." https://www.o-ran.org/software (accessed Jan. 16, 2026)

[6]. O-RAN Working Group 11, "Study on Security for Near Real Time RIC and xApps," O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v06.00, Oct. 2025

[7]. "The Falco Project," Falco. https://falco.org/docs/

[8]. "Keylime: Bootstrap & Maintain Trust on the Edge/Cloud and IoT," https: //keylime.dev/, Keylime Project

References

[9]. C.-F. Hung, Y.-R. Chen, C.-H. Tseng, and S.-M. Cheng, “Security Threats to xApps Access Control and E2 Interface in O-RAN,” IEEE Open Journal of the Communications Society, vol. 5, pp. 1197–1203, 2024

[10]. P. Kuure, “Security and Trust in O-RAN,” Master’s thesis, South-Eastern Finland University of Applied Sciences, Finland, 2024

[11]. O-RAN Working Group 11, “O-RAN Security Threat Modeling and Risk Assessment,” O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v07.00, Oct. 2025

[12]. Cloud Native Computing Foundation, “The spiffe project: Secure production identity framework for everyone,” https://spiffe.io, 2024

Thank You